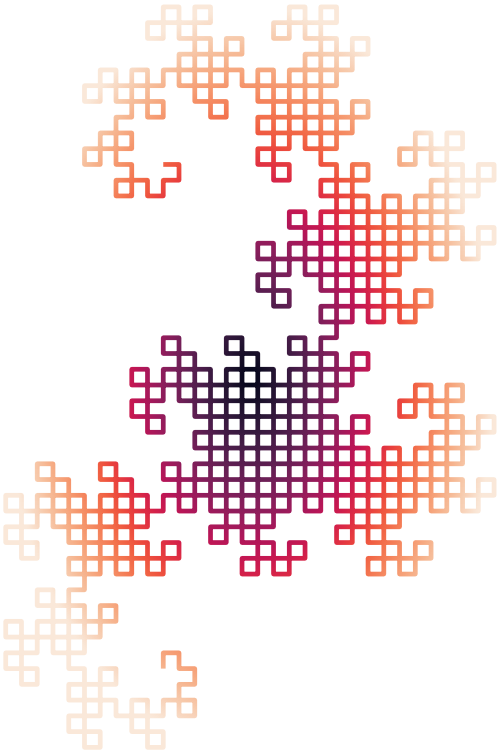
MASTER OF SCIENCE
IN ENGINEERING

Hes·so
Haute Ecole Spécialisée
de Suisse occidentale
Fachhochschule Westschweiz
University of Applied Sciences and Arts
Western Switzerland

Environment & Kernel Programming

Lab 01: Embedded Linux Environment
&
Lab 02: Linux Kernel Programming



HES-SO Master
Computer Science - Embedded
CSEL - Jacques Supcik
Spring Semester 2026

Yann Sierro, Rémi Heredero
01.04.2026 - v0.1.0 - final

Contents

1 Embedded Linux Environment . . . 3
1.1 Questions . . . 3
1.1.1 How to generate U-Boot? . . . 3
1.1.2 How to add and build a additional package in Buildroot? . . . 3
1.1.3 How to modify the Linux kernel configuration? . . . 4
1.1.4 How to generate a custom rootfs? . . . 4
1.1.5 How to use the eMMC card instead of the SD card? . . . 4
1.1.6 In cours support, we find several configurations of the development environment. What would be the optimal configuration for developing only user-space applications? . . . 4
2 Linux Kernel Programming . . . 5
2.1 Cheatsheet commands . . . 5
2.2 Exercises . . . 5
Glossary . . . 11

The repository for this labs can be found at the following address:

https://github.com/Klagarge/MSE-MA-CSEL

1 Embedded Linux Environment

In this laboratory, we see how to setup our environnement and how to have several way to boot. That include a **`boot.cifs`** that allow us to load the rootfs from samba to easily share the rootfs between the host and the target. And also a **`boot.tftp`** that allow us to load the kernel by tftp, which is really usefull when we want to modify the kernel and test it without having to reflash the whole system.

We also see how to debug our system with a remote debugger. That allow us to use debug in our code editor (vscode) a programm that run on the target.

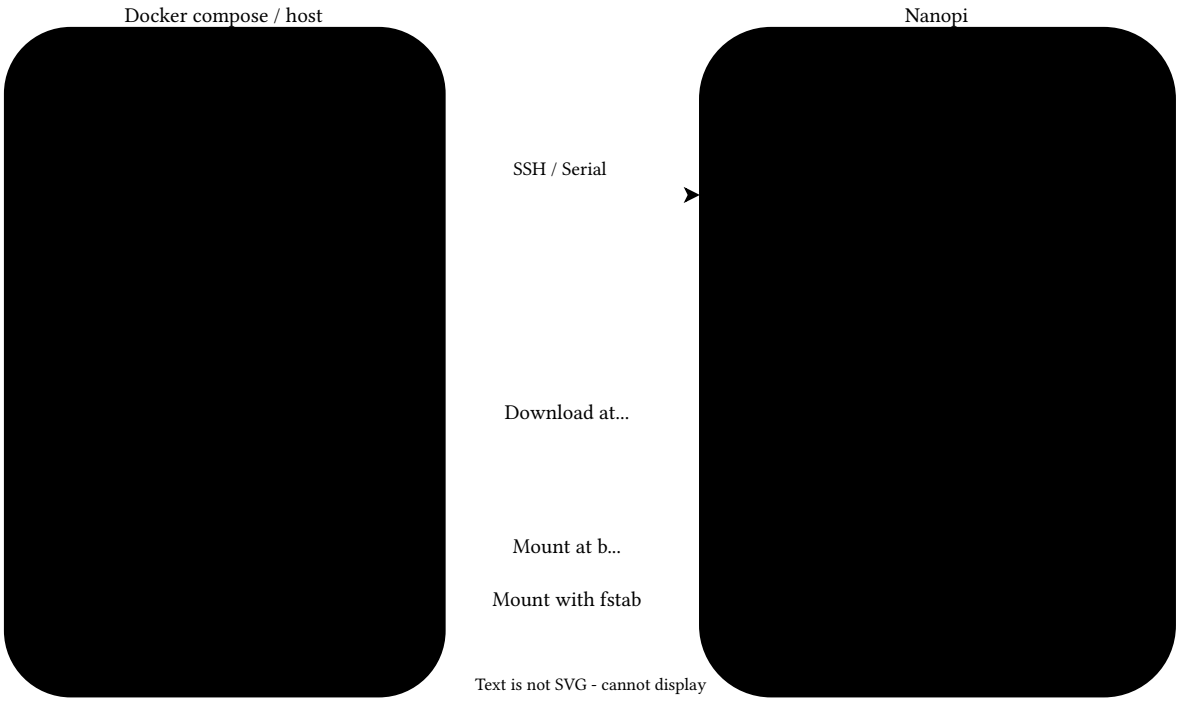


Figure 1 - Development environment schema

1.1 Questions

1.1.1 How to generate U-Boot?

We use buildroot, a tool to build embedded Linux. It can generate the U-Boot bootloader. With **`make menuconfig`**, we can select the U-Boot package. U-boot can be configured with **`make uboot-menuconfig`**

And finally, when we have configured everything, we can build the whole system with **`make`** command. Or only uboot with **`make uboot`** command.

1.1.2 How to add and build a additional package in Buildroot?

In buildroot, with **`make menuconfig`**, we can select the package we want in **`Target packages`** section. We can specifically build it with **`make <package-name>`** command. Otherwise, it will be built with the whole system when we run **`make`** command.

1.1.3 How to modify the Linux kernel configuration?

Like all package, with **`make <package-name>-menuconfig`** command. So, for Linux kernel:

```
|> make linux-menuconfig
```

1.1.4 How to generate a custom rootfs?

First of all, select the type of filesystem you want to generate in **`Filesystem images`** section of **`make menuconfig`**. We can use an overlay to customise our rootfs. The overlay is a directory (inin the board folder) with the same structure as rootfs and it will merge with the generated rootfs. So, we can add files and directories in the overlay and they will be added to the final rootfs.

1.1.5 How to use the eMMC card instead of the SD card?

We need to change the boot script **`(boot*.cmd)`** to load from eMMC by changing the **`fatload`** command with the correct number. Probably 1 instead of 0.

```
fatload mmc 1 $kernel_addr_r Image
```

1.1.6 In cours support, we find several configurations of the development environment. What would be the optimal configuration for developing only user-space applications?

If we develop only user space program, we don't need to load kernel by tftp. But it's really usefull to have rootfs load by samba. So the best approach is to use the **`boot.cifs`**.

2 Linux Kernel Programming

2.1 Cheatsheet commands

- **`modinfo <module.ko>`**: display information about a kernel module
- **`insmod <module.ko>`**: install a kernel module (without checking for dependencies)
- **`rmmod <module.ko>`**: uninstall a kernel module
- **`lsmod`**: list the currently loaded kernel modules
- **`dmesg`**: display the kernel log
- **`cat /proc/modules`**: display the currently loaded kernel modules with more details
- **`modprobe <module>`**: install a kernel module and its dependencies
- **`modprobe -r <module>`**: uninstall a kernel module and its dependencies
- **`make`**: build the kernel module
- **`make install`**: install the kernel module in the root filesystem

2.2 Exercises

Task A — Generate kernel module out of tree

1. Create the skeleton of a kernel module and generate it outside the kernel sources using a Makefile. The module should display a message when it is registered and when it is uninstalled.

We already have a skeleton in **`src/02-modules/exercice01`** (now **`solutions/02_modules/exercice01`** that we move to **`src/01-skeleton`**). We see on the Makefile that the module is generated outside the kernel sources with the **`KDIR`** variable imported from **`src/kernel_settings`**. This variable point to the kernel sources. The Makefile also use the **`PWD`** variable to the current directory. The **`make`** command will use these variables to generate the module in the current directory.

```
$(MAKE) -C $(KDIR) M=$(PWD) ARCH=$(CPU) CROSS_COMPILE=$(TOOLS) modules
```

2. Test on the host machine the command modinfo1 on your module skeleton and compare the information returned with that of the source code.

```
|> modinfo mymodule.ko
filename:       /workspace/src/01-skeleton/mymodule.ko
license:        GPL
description:    Module skeleton
author:         Klagarge <remi@heredero.ch>
author:         Fastium <fastium.pro@proton.me>
depends:
name:           mymodule
vermagic:       5.15.148 SMP preempt mod_unload aarch64
parm:           text:charp
parm:           elements:int
```

3. Install the module (insmod) and check the kernel log (dmesg)

```
|> insmod mymodule.ko
[ 1727.896902] mymodule: loading out-of-tree module taints kernel.
[ 1727.903442] Linux module 01 skeleton loaded
```

We can see the module is indead out-of-tree and correctly loaded.

```
|> dmesg | tail -5
[ 1381.694764] CIFS: Attempting to mount \\192.168.53.4\workspace
[ 1727.896902] mymodule: loading out-of-tree module taints kernel.
[ 1727.903442] Linux module 01 skeleton loaded
[ 1727.907659]   text: dummy text
[ 1727.907659]   elements: 1
```

4. Compare the results obtained by the lsmod command with those obtained with the cat /proc/modules command

```
|> lsmod
Module                  Size  Used by    Tainted: G
mymodule               16384  0
...

|> cat /proc/modules
mymodule 16384 0 - Live 0xffff8000011bf000 (O)
...
```

The **`/proc/modules`** file give us more details about the state of the module. We see it is now live (charged in memory and running)

5. Uninstall the module (rmmod).

```
|> rmmod mymodule.ko
[ 2989.535793] Linux module skeleton unloaded
```

6. Adapt the Makefile of the module to allow the installation of the module with other kernel modules allowing the use of the modprobe command. The module should be installed in the root filesystem used in cifs by the target.

```
# On host:
|> make install

# On target:
|> modprobe mymodule
[ 3359.811183] Linux module 01 skeleton loaded
```

Task B — Adapt the kernel module to receive parameters

Adapt the kernel module of the previous exercise to receive two or three parameters of your choice. These parameters will be displayed in the console when the module is loaded.

```
|> modprobe mymodule
[ 3583.616662] Linux module skeleton ex02 loaded
|> dmesg | tail -5
[ 3559.279143]   number: 1
[ 3581.198562] Linux module skeleton unloaded
[ 3583.616662] Linux module skeleton ex03 loaded
[ 3583.621085]   text: The answer to the Ultimate Question of Life, The Universe, and
Everything
[ 3583.621085]   number: 42
|> modprobe -r mymodule
[ 3588.404778] Linux module skeleton unloaded
```

Task C — What does it mean the 4 values in `/proc/sys/kernel/printk` ?

We can show what there is in:

```
|> cat /proc/sys/kernel/printk
7 4 1 7
```

The number specified the level of output in a console.

This file specifies the log level for:
current (7), default (4), minimum (1) and boot-time default (7).

This number matches with this table (printk documentation):

Name	String	Alias function
KERN_EMERG	“0”	pr_emerg()
KERN_ALERT	“1”	pr_alert()
KERN_CRIT	“2”	pr_crit()
KERN_ERR	“3”	pr_err()
KERN_WARNING	“4”	pr_warning()
KERN_NOTICE	“5”	pr_notice()
KERN_INFO	“6”	pr_info()
KERN_DEBUG	“7”	pr_debug() and pr_devel() if DEBUG is defined
KERN_DEFAULT	“”	
KERN_CONT	“c”	pr_cont()

Task D — Create module with dynamic allocation and a chained list

Create dynamically elements in the kernel. Adapt a kernel module to specify at the installation the number of element to create a initial text. Each element will contain a unique number. The elements are create at the installation of the module adn chained in a list. These elements will be destruct during the uninstallation of the module. Some information messages are emits to allow debugging.

To allocate memory in the kernel, we can use the **`kcalloc`** function. It allows to allocate directly the memory for all element. It's also possible to use **`kzalloc`** in a loop to allocate memory for each element. We prefer allocate all the memory at once to avoid fragmentation and to be sure all the memory can be allocated.

```
struct element* element_ptr = kcalloc(elements, sizeof(struct element), GFP_KERNEL);

for (int i = 0; i < elements; i++) {
  struct element* e = element_ptr + i;
      if (e != 0) {
          strncpy(e->text, text, TEXT_LENGTH_MAX - 1);
          e->unique_number = i;
          list_add_tail(&e->node, &list_unique_elements);
    pr_info ("add element %d: %s\n", e->unique_number, e->text);
      }
  }
```

Task E — Display the processor chip ID, CPU temperature and the MAC adress of the Ethernet controller

- *Chip ID registers:* 0x01c1′4200 *to* 0x01c1′420c
- *32 bits register of the temperature sensor:* 0x01c2′5080
- *two 32 bits registers of the Ethernet controller MAC address:* 0x01c3′0050 *and* 0x01c3′0054

To calculate the temperature value, there is this formul:

$$\text{temperature} = -1991 \cdot \frac{\text{register value}}{10} + 223000 \tag{1}$$

The chip ID can be verified in ***`/proc/iomem`****. The register value of the temperature can be verified in the file:* ***`/sys/class/thermal/thermal_zone0/temp`****. The MAC address can be verified with* ***`ifconfig`****.*

The resources are savec in a struct:

```
static struct resource* resources[3] = {[0] = 0,};
```

resources[0] is reserved for the chip ID, resources[1] for the temperature sensor and resources[2] for the Ethernet controller.

We first allocate the resources with **`request_mem_region`** function. Then we can map the physical address to a virtual address with **`ioremap`** function. Finally, we can read the value of the registers with **`ioread32`** function. The request fail because we have an overlap with the EEPROM, but we can ignore this error because we can still read the registers with **`ioremap`** function.

```
// Request the resource at (CHIP_ID_BASE_ADDR)
resources[0] = request_mem_region(CHIP_ID_BASE_ADDR, 0x1000, "nanopi - chip ID");

// Map the physical address (CHIP_ID_BASE_ADDR) to a virtual address (registers[0])
registers[0] = ioremap(CHIP_ID_BASE_ADDR, 0x1000);
```

Task F — Kernel thread

Develop a module which allows to instanciate a thread in the kernel. This thread will display a message every 5 seconds. Use the function ***`(5)`*** *to sleep the thread from* ***`linux/delay.h`****.*

Easy exercice, a thread in the kernet is a **`struct task_struct*`** that can be created with **`kthread_run`**

Task G — Sleeping

Develop a module which instanciate 2 threads in the kernel. The first one will wait a wake up notification from the second thread and will sleep. The second will send the notification every 5 seconds. Then it will sleep. We will use the waitqueue for the sleeping function. To allow debugging, each thread will send a message when it wakes up.

This exercice make 2 threads in concurrency with wait queue. Here the queue ware declare statically with the macro **`DECLARE_WAIT_QUEUE_HEAD`**. Then for this exercice we use an atomic trigger with 2 queues. It important that the trigger is atomic or protected by mutex because there is concurrency. The wait queues are used to wait until the trigger has changed to keep synchronization between the threads.

It is very important to add **`kthread_should_stop()`** as a condition to wake up queue, because if there is a problem during the implementation, we cannot kill the code.

Task H — Interrupts

Develop a module which allows to detect every push on the button of the nanopi with interrupt. Every interrupts will send a message for debugging.

- *Use the service* ***`gpio_request(<io_nr>, <label>)`***
- *Get the interrupt vector with* ***`gpio_to_irq(<io_nr>)`***
- *Extension card information:*
  - *k1 - gpio: A, pin_nr=0, io_nr=0*
  - *k2 - gpio: A, pin_nr=2, io_nr=2*
  - *k3 - gpio: A, pin_nr=3, io_nr=3*

Glossary